Лабораторная работа 12. Drift и отложенное качество модели

Версия: 2026–2027 учебный год Формат: команда 2–5 человек с индивидуальной сдачей Продолжительность: 2 академических часа и самостоятельная работа Предшествующая работа: ЛР11 «Наблюдаемость ML-сервиса»

1. Цель

Построить воспроизводимый мониторинг данных и качества модели: сравнить reference/current окна, сформировать drift report, корректно присоединить задержанные labels, оценить coverage и качество, а затем принять управляемое решение по version-controlled policy.

2. Результаты обучения

Каждый студент должен уметь:

- различить drift, data quality incident и quality degradation;
- объяснить границы reference/current окна;
- воспроизвести PSI и один его риск;
- рассчитать mature cohort и label coverage;
- связать запись с model version и release manifest;
- интерпретировать ROC AUC/F1 только при достаточных labels;
- выполнить отказной сценарий и применить runbook;
- обосновать `hold`, `investigate`, `incident_review` или `retraining_review`.

3. Командный результат

Команда предоставляет tag `lr12-v1` со следующими артефактами:

1. `monitoring/drift_policy.json`;
2. модуль расчёта drift и delayed quality;
3. reference/current datasets с явными временными границами;
4. prediction log и label log с уникальным `request_id`;
5. JSON-отчёт с PSI, coverage, metrics и decision;
6. минимум четыре автоматических теста;
7. runbook реакции;
8. один общий baseline report и отдельный отчёт каждого участника;
9. issue, небольшие PR, review и индивидуальные evidence cards.

4. Проверенная версия расширения

Обязательный расчёт использует pandas и NumPy из starter-project. Дополнительно разрешён Evidently `>=0.7.23,<0.8`, проверенный 20 октября 2026 года. Если команда использует Evidently, она сохраняет точную версию, конфигурацию, JSON и объясняет различия с прозрачным baseline. Один HTML без policy и машинно-читаемого результата не принимается.

5. Подготовка

```
python -m pip install -e ".[dev]"
python -m pytest tests/test_monitoring.py -q
```

Для дополнительного Evidently report:

```
python -m pip install -e ".[monitoring]"
```

Не публикуйте production-данные, персональные сведения, токены и сырые чувствительные признаки.

6. Входные контракты

Reference/current содержат `DepHour` и `IsWeekend`. Prediction log содержит:

Поле	Требование
`request_id`	уникальный идентификатор
`event_time`	UTC timestamp прогноза
`prediction_probability`	число от 0 до 1
`model_version`	точная версия из release manifest
признаки	только разрешённые policy поля

Label log содержит `request_id`, `label_time`, `ArrDelay`. Цель определяется как `ArrDelay >= 15`. Join должен быть one-to-one.

7. Reference и current окна

Команда документирует источник, начало, конец, фильтры и число строк обоих окон. Reference должен предшествовать current либо соответствовать утверждённой validation-выборке champion. Запрещено подбирать reference после просмотра current только ради прохождения threshold.

8. Проверка качества данных до drift

До статистики проверьте схему, типы, пропуски, диапазон `DepHour`, бинарность `IsWeekend`, уникальность идентификаторов и timestamps. Нарушение контракта классифицируется отдельно. Нельзя маскировать значение 900 широкими bins и называть это естественным drift.

9. Числовой PSI

Для `DepHour` постройте quantile bins только по reference. Примените те же границы к current, сгладьте нулевые доли и сохраните score. В отчёте укажите method `psi_reference_bins`, warning, alert и число строк. Студент должен уметь объяснить влияние числа bins.

10. Категориальный PSI

Для `IsWeekend` используйте объединение категорий reference и current. Новая категория не должна исчезать из расчёта. Сохраните score и статус. Для реального высококардинального признака команда обязана согласовать агрегацию редких категорий.

11. Policy

Минимальная policy содержит:

```
{
"minimum_window_rows": 8,
"psi_warning": 0.1,
"psi_alert": 0.25,
"expected_label_delay_hours": 24,
"minimum_labeled_rows": 8,
"minimum_label_coverage": 0.75,
"minimum_roc_auc": 0.7,
"minimum_f1": 0.6,
"automatic_retraining": false
}
```

Учебные значения можно изменить до запуска с письменным обоснованием. Подгонять пороги после получения результата запрещено.

12. Базовый запуск

```
python -m mlops_flight_delay.monitoring \
--reference data/monitoring/reference.csv \
--current data/monitoring/current.csv \
--predictions data/monitoring/predictions.csv \
--labels data/monitoring/labels.csv \
--policy monitoring/drift_policy.json \
--as-of 2026-10-20T12:00:00Z \
```

```
--output reports/monitoring/lr12-team-baseline.json
```

Отчёт должен воспроизводиться из tag без ручного редактирования результата.

13. Mature cohort

В coverage включаются прогнозы с `event_time <= as_of - expected_label_delay`. Свежие события исключаются из знаменателя. В отчёте обязательны `as_of`, `eligible_cutoff`, число eligible и число matched labels. Все времена должны содержать часовой пояс.

14. Coverage gate

Если matched rows меньше `minimum_labeled_rows` или coverage ниже порога, статус — `insufficient_evidence`. Метрики качества не используются для решения о retraining. Команда должна показать, что отсутствие метки и отрицательная метка — разные состояния.

15. Метрики качества

При достаточных labels рассчитайте ROC AUC, precision, recall и F1 по одной model version. Если в окне один класс, ROC AUC не подменяется нулём: отчёт получает `insufficient_evidence`. Threshold классификации фиксируется до расчёта.

16. Сегментный анализ

Добавьте минимум один безопасный разрез, например `IsWeekend`. Для каждого сегмента укажите N и coverage. Не делайте вывод по малому сегменту. Индивидуальный участник может отвечать за отдельный сегмент, но должен использовать общую policy и командный release.

17. Decision gate

Starter-project реализует следующие решения:

Условие	Действие
labels недостаточно	`hold`
drift alert, качество в норме	`investigate`
quality breach без drift alert	`incident_review`
drift alert и quality breach	`retraining_review`
пороги не нарушены	`no_action`

`retraining_review` создаёт процесс проверки candidate, но не назначает его champion автоматически.

18. Отказные сценарии

Каждый участник выбирает собственный сценарий:

Сценарий	Ожидаемый результат
удалить зрелые labels	coverage падает, решение `hold`
дублировать `request_id`	явная ошибка one-to-one контракта
сдвинуть `as_of`	изменяется mature cohort
добавить недопустимый `DepHour`	data quality incident до drift
оставить один класс	ROC AUC не рассчитывается
изменить только current distribution	`investigate`, а не auto retraining

Работайте в отдельной ветке или копии входных данных. Общий baseline не портится.

19. Runbook

Используйте `docs/runbooks/model-monitoring.md`. Зафиксируйте первые проверки, владельца, действие, условие завершения и ссылку на evidence. Команда не должна ограничиваться фразой «переобучить».

20. Postmortem

Для одного общего сценария создайте краткий postmortem: временная линия, влияние, первопричина, что помогло обнаружить, что задержало реакцию, corrective action и владелец. Учебный инцидент явно маркируется как моделирование.

21. Автоматические тесты

Обязательны проверки:

1. drift обнаруживается на изменённом current;
2. bins формируются без использования current;
3. свежие прогнозы исключаются из coverage;
4. дубликат `request_id` отклоняется;
5. insufficient labels дают `hold`;
6. retraining review требует одновременно drift и quality breach;
7. JSON parseable и содержит policy version;
8. линтер и весь pytest проходят.

22. CI

Workflow запускает тесты и базовую команду мониторинга, сохраняет JSON в diagnostic artifacts. Evidently UI в CI поднимать не требуется. Floating dependency без upper bound не принимается.

23. Командные роли

Для команды 2–5 человек:

- владелец окон и data quality;
- владелец drift metric и статистических ограничений;
- владелец delayed-label join и coverage;
- владелец quality gate и тестов;
- владелец runbook/postmortem и release correlation.

В малой команде роли совмещаются, но артефакты различаются. Роли ротируются относительно ЛР11.

24. Индивидуальная сдача участника команды

Командный результат: tag `lr12-v1`, policy, тесты, baseline JSON, runbook и postmortem.

Каждый участник отдельно заполняет `docs/evidence/lr12-evidence-card.md`. Совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда и GitHub-логин
Лабораторная	ЛР12 и tag `lr12-v1`
Роль	роль и отличие от роли в ЛР11
Задача	собственная issue с наблюдаемым критерием
Личный артефакт	metric, window, join, test, policy, report, runbook или postmortem
Запуск	личные параметры, `as_of` и отдельный JSON
Evidence	score, coverage, metric, decision и model version
Решение	альтернатива, риск и проверяемое обоснование
Использование ИИ	инструмент и самостоятельно проверенные утверждения

25. Минимальный допуск

1. Командная версия зафиксирована tag и commit SHA.
2. Участнику назначена собственная issue.

3. Есть содержательный личный артефакт и отдельный JSON-отчёт.
4. Изменение прошло PR и review другого участника.
5. Evidence card содержит рабочие ссылки.
6. CI успешен либо исключение письменно принято преподавателем.
7. Студент может объяснить окно, знаменатель coverage и принятое действие.

26. Индивидуальная защита

За 3–4 минуты участник:

1. показывает свой артефакт в tag;
2. воспроизводит личный сценарий;
3. объясняет reference/current и mature cohort;
4. находит model version и policy version в отчёте;
5. интерпретирует один PSI и одну quality metric;
6. выполняет выбранное преподавателем изменение `as_of`, threshold или входного файла;
7. объясняет, почему решение не означает автоматический выпуск.

27. Повторная сдача и стоп-факторы

Повторная сдача выполняется через переоткрытую issue, новый PR, обновлённый отчёт и evidence card. История первой попытки сохраняется.

Стоп-факторы:

- current использован для построения reference bins;
- свежие прогнозы ошибочно включены как missing labels;
- метрики рассчитаны при недостаточном coverage;
- model version или временные границы отсутствуют;
- drift автоматически запускает promotion;
- HTML заменяет policy и JSON;
- общий отчёт выдан за личный;
- студент не может объяснить собственное решение.

28. Критерии оценивания

Критерий	Баллы	Полное выполнение
Окна, контракт и drift	2	воспроизводимые границы, корректный PSI и ограничения
Delayed labels и coverage	2	mature cohort, one-to-one join и достаточность evidence
Quality и decision gate	2	метрики корректны, auto retraining отсутствует
Тесты, CI и runbook	2	отказные сценарии воспроизводимы, реакция завершена
Индивидуальный вклад и защита	2	личный отчёт, артефакт, review и объяснение

Максимум: 10 баллов. Рекомендуемый порог зачёта — 6 баллов при отсутствии стоп-факторов.

29. Чек-лист

- [ ] Команда состоит из 2–5 человек, роли ротированы.
- [ ] Reference/current границы и model version зафиксированы.
- [ ] Data quality проверена до drift.

- [ ] Числовые bins построены только по reference.
- [ ] Mature cohort использует expected label delay.
- [ ] Coverage содержит числитель и знаменатель.
- [ ] Quality не считается при недостаточных labels.
- [ ] Decision соответствует policy.
- [ ] Автоматическое promotion отсутствует.
- [ ] Runbook и postmortem содержат владельцев.
- [ ] Тесты и CI успешны.
- [ ] У каждого участника отдельный отчёт и evidence card.
- [ ] Итоговая версия отмечена tag `lr12-v1`.

30. Связь с ЛР13

В ЛР13 временная корректность распространяется на признаки: студент сравнит наивный join с point-in-time join. Ошибка available time способна создать leakage в обучении и исказить drift, поэтому timestamps и контракт ЛР12 сохраняются.

31. Официальные источники

- Evidently PyPI: https://pypi.org/project/evidently/
- Evidently API `Report` и `DataDriftPreset`: https://evidentlyai.github.io/evidently/api-reference/v0.7.19/evidently.html
- Evidently presets API: https://evidentlyai.github.io/evidently/api-reference/main/evidently/presets.html
- scikit-learn classification metrics: https://scikit-learn.org/stable/api/sklearn.metrics.html